

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0611 | COURSE NAME: MACHINE LEARNING

CO5 – CONSTRAINT BASED PROBLEM SOLVING

Code of Conduct

I **B. HEMANTH KUMAR** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Question 1

You are given a large dataset of patient health records stored in a CSV file, containing missing values, inconsistent formats, and outliers. Using NumPy and Pandas, design a constraint-based data cleaning pipeline that ensures: (i) no missing values remain, (ii) all numerical values fall within medically valid ranges, and (iii) categorical fields are standardized. Explain how you will use indexing, filtering, and aggregation functions to enforce these constraints while preserving maximum data integrity.

Answer

A constraint-based data cleaning pipeline can be designed using Pandas and NumPy to identify and correct missing values, invalid numerical values, inconsistent categorical values, and outliers.

Step 1: Load the Dataset

```
import pandas as pd
import numpy as np

df = pd.read_csv("patient_health_records.csv")
print(df.head())
print(df.info())
```

The `read_csv()` function loads the patient records into a Pandas DataFrame.

Step 2: Identify Missing Values

```
print(df.isnull().sum())
```

```
df["Age"] = df["Age"].fillna(df["Age"].median())
df["BloodPressure"] = df["BloodPressure"].fillna(
    df["BloodPressure"].median()
)
df["Gender"] = df["Gender"].fillna("Unknown")
```

Numerical missing values can be replaced using the median, while categorical missing values can be replaced with a suitable category such as “Unknown.”

Step 3: Standardize Categorical Fields

```
df["Gender"] = df["Gender"].astype(str).str.strip().str.lower()

df["Gender"] = df["Gender"].replace({
    "m": "male",
    "f": "female"
})
```

This ensures that the same category is represented consistently.

Step 4: Apply Medically Valid Constraints

```
df = df[(df["Age"] >= 0) & (df["Age"] <= 120)]

df = df[
    (df["Temperature"] >= 35) &
    (df["Temperature"] <= 42)
]

df = df[
    (df["HeartRate"] >= 30) &
    (df["HeartRate"] <= 220)
]
```

These constraints remove or isolate records containing medically invalid values.

Step 5: Detect Outliers

```
Q1 = df["Age"].quantile(0.25)
```

```
Q3 = df["Age"].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower = Q1 - 1.5 * IQR
```

```
upper = Q3 + 1.5 * IQR
```

```
df = df[(df["Age"] >= lower) & (df["Age"] <= upper)]
```

The IQR method helps detect extreme observations.

Step 6: Use Indexing and Filtering

```
print(df.loc[:, ["Age", "Gender", "HeartRate"]])
```

```
valid_records = df[
    (df["Age"] >= 0) &
    (df["Age"] <= 120)
]
```

Pandas indexing accesses specific records or columns, while filtering retains only records satisfying the constraints.

Step 7: Validate the Constraints

```
print(df.isnull().sum())
```

```
print(df["Gender"].unique())
```

```
print(df.describe())
```

Aggregation functions such as `mean()`, `median()`, `min()`, and `max()` can be used to verify whether numerical values are within expected ranges.

Conclusion

The pipeline uses Pandas for data loading, indexing, filtering, grouping, and cleaning, while NumPy supports numerical calculations and statistical operations. Missing values, invalid values, outliers, and inconsistent categories are systematically handled. This preserves maximum data integrity while ensuring that the final dataset satisfies the defined constraints.

Question 3

Two large Pandas DataFrames containing customer transaction and demographic data must be combined. However, constraints include: (i) no duplication of customer IDs, (ii) consistent indexing after merging, and (iii) preservation of all valid records. Describe how you will perform the combining operation, validate constraints using filtering and grouping, and ensure correctness through sorting and indexing techniques. Include steps to debug mismatched or missing entries.

Answer

Two DataFrames can be combined using Pandas `merge()`, `concat()`, or `join()` functions. Since `CustomerID` is the common key, `merge()` is the most appropriate method.

Step 1: Load the DataFrames

```
import pandas as pd

transactions = pd.read_csv("transactions.csv")
demographics = pd.read_csv("demographics.csv")
```

Step 2: Check Customer IDs

```
print(transactions["CustomerID"].duplicated().sum())
print(demographics["CustomerID"].duplicated().sum())

transactions = transactions.drop_duplicates(subset="CustomerID")
demographics = demographics.drop_duplicates(subset="CustomerID")
```

Step 3: Merge the DataFrames

```
merged_df = pd.merge(
    transactions,
    demographics,
    on="CustomerID",
    how="outer",
    indicator=True
)
```

The outer merge preserves valid records from both DataFrames. The indicator column identifies whether each record occurs in both datasets or only one.

Step 4: Check Mismatched Records

```
print(
    merged_df[merged_df["_merge"] != "both"]
)
```

Step 5: Check for Missing Values

```
print(merged_df.isnull().sum())
```

Missing demographic information should be investigated rather than automatically deleting the affected customer record.

Step 6: Validate Duplicate Customer IDs

```
duplicates = merged_df[
    merged_df["CustomerID"].duplicated(keep=False)
]

print(duplicates)

assert not merged_df["CustomerID"].duplicated().any()
```

Step 7: Reset the Index

```
merged_df = merged_df.sort_values(
    "CustomerID"
).reset_index(drop=True)
```

Sorting organizes the merged records, while `reset_index()` creates a consistent sequential index.

Step 8: Use Grouping for Validation

```
customer_counts = merged_df.groupby(
    "CustomerID"
).size()

print(customer_counts)
```

Step 9: Debug Mismatches

```
transaction_ids = set(transactions["CustomerID"])
demographic_ids = set(demographics["CustomerID"])

print("Missing in demographics:",
```

```
transaction_ids - demographic_ids)
```

```
print("Missing in transactions:",  
      demographic_ids - transaction_ids)
```

Conclusion

The combination process uses `merge()` with `CustomerID` as the key. Duplicate IDs are checked, mismatches are identified using set operations and the merge indicator, and grouping validates record counts. Finally, sorting and `reset_index()` provide consistent indexing while an outer merge ensures that valid records from both datasets are preserved.

Question 4

You are tasked with computing statistical measures (mean, median, variance) on a dataset using NumPy and Pandas, but with constraints on both computational efficiency and numerical accuracy. Explain how you will implement aggregation and statistical functions while avoiding precision loss and redundant computations. Discuss how filtering and indexing can be used to exclude irrelevant data points, and how you would debug incorrect statistical outputs.

Answer

NumPy and Pandas provide efficient statistical functions for calculating mean, median, and variance. The data should first be cleaned and filtered so that irrelevant or invalid values do not affect the results.

Step 1: Load the Data

```
import pandas as pd  
import numpy as np
```

```
df = pd.read_csv("data.csv")
```

Step 2: Select the Required Column

```
values = df["Value"].dropna()
```

Indexing ensures that only the required column is processed.

Step 3: Filter Irrelevant Values

```
values = values[values >= 0]
```

```
values = values[  
    (values >= 0) &
```

```
(values <= 100000)
]
```

Step 4: Calculate Statistical Measures

```
mean_value = values.mean()
median_value = values.median()
variance_value = values.var()
```

```
mean_np = np.mean(values)
median_np = np.median(values)
variance_np = np.var(values)
```

Pandas and NumPy functions are vectorized and therefore more efficient than manually iterating through every record.

Step 5: Avoid Redundant Calculations

```
valid_values = df.loc[
    df["Value"].notna() &
    (df["Value"] >= 0),
    "Value"
]
```

```
mean_value = valid_values.mean()
median_value = valid_values.median()
variance_value = valid_values.var()
```

Step 6: Improve Numerical Accuracy

```
values = values.astype(np.float64)
```

Using float64 provides better precision for statistical calculations than lower-precision numerical types.

Step 7: Debug Incorrect Results

```
print("Number of values:", len(values))
print(values.describe())
print(values.isnull().sum())
print(values.dtype)
```

```
print(np.mean(values))
print(values.mean())
```

```
print(np.var(values))  
print(values.var())
```

```
print(np.var(values, ddof=1))
```

A difference in variance may occur because Pandas uses sample variance by default (ddof=1), while NumPy uses population variance by default (ddof=0). The ddof parameter can be adjusted to make the calculations consistent.

Conclusion

Efficient statistical analysis is achieved by filtering the dataset once, selecting only required columns, using vectorized NumPy/Pandas functions, and avoiding repeated calculations. float64 helps maintain numerical precision. Debugging should include checking missing values, data types, observation counts, ranges, and differences in statistical definitions.

Question 5

Design a complete data analysis pipeline that reads multiple data files, processes them using Pandas DataFrames, and generates visualizations. Constraints include: (i) efficient file I/O operations, (ii) correct grouping and sorting of data, (iii) filtering based on user-defined conditions, and (iv) meaningful plotting outputs. Explain how each stage (loading, processing, aggregation, and plotting) will be implemented and optimized, along with debugging steps for handling corrupted files or inconsistent data formats.

Answer

A complete data analysis pipeline can be developed using Pandas, NumPy, and Matplotlib. The pipeline consists of file loading, validation, cleaning, filtering, grouping, sorting, aggregation, and visualization.

Step 1: Import Libraries

```
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt
```

Step 2: Read Multiple Files

```
files = [  
    "sales1.csv",  
    "sales2.csv",
```



```

    "sales3.csv"
]

dataframes = []

for file in files:
    try:
        df = pd.read_csv(file)
        dataframes.append(df)
    except Exception as e:
        print("Error reading", file, ":", e)

```

Step 3: Combine the Files

```

data = pd.concat(
    dataframes,
    ignore_index=True
)

```

ignore_index=True creates a consistent index after combining the DataFrames.

Step 4: Inspect and Validate the Data

```

print(data.head())
print(data.info())
print(data.isnull().sum())
print(data.dtypes)

```

Step 5: Standardize Data Formats

```

data["Product"] = (
    data["Product"]
    .astype(str)
    .str.strip()
)

data["Sales"] = pd.to_numeric(
    data["Sales"],
    errors="coerce"
)

```

errors='coerce' converts invalid numerical values into NaN, which can then be handled.

Step 6: Handle Missing Values

```
data["Sales"] = data["Sales"].fillna(0)
```

The appropriate strategy depends on the meaning of the missing data.

Step 7: Apply User-Defined Filtering

```
filtered_data = data[
    data["Sales"] > 10000
]
```

```
filtered_data = data[
    (data["Sales"] > 10000) &
    (data["Quantity"] > 5)
]
```

Step 8: Group the Data

```
sales_summary = filtered_data.groupby(
    "Product"
)["Sales"].sum()
```

Step 9: Sort the Results

```
sales_summary = sales_summary.sort_values(
    ascending=False
)
```

Step 10: Generate Meaningful Visualization

```
sales_summary.plot(
    kind="bar",
    figsize=(10, 5)
)
```

```
plt.title("Sales by Product")
plt.xlabel("Product")
plt.ylabel("Total Sales")
plt.tight_layout()
plt.show()
```

A bar chart is appropriate for comparing sales among different products.

```
data.plot(  
    x="Date",  
    y="Sales",  
    kind="line",  
    figsize=(10, 5)  
)
```

```
plt.title("Sales Trend")  
plt.xlabel("Date")  
plt.ylabel("Sales")  
plt.tight_layout()  
plt.show()
```

A line chart can be used for time-series data.

Step 11: Debug Corrupted Files

```
try:  
    df = pd.read_csv("sales.csv")  
except pd.errors.ParserError:  
    print("File contains formatting errors.")  
except FileNotFoundError:  
    print("File not found.")  
except Exception as e:  
    print("Unexpected error:", e)
```

Step 12: Debug Inconsistent Formats

```
print(data.dtypes)  
print(data.columns)  
print(data.isnull().sum())  
print(data.nunique())
```

Step 13: Validate the Final Output

```
assert not data.empty  
print(data.describe())  
print(data.head())
```

Pipeline Summary

File Input → Data Validation → Cleaning → Combining → Filtering → Grouping → Sorting → Aggregation → Visualization → Validation

Conclusion

The proposed pipeline efficiently processes multiple files using Pandas. `concat()` combines datasets, filtering applies user-defined conditions, `groupby()` performs aggregation, and `sort_values()` organizes the results. Matplotlib provides meaningful visualizations. Error handling, data-type checking, missing-value detection, and validation ensure that corrupted or inconsistent data does not produce unreliable results.